Session 2: Semantic Types and Type-Driven Interpretation

What Can Combine With What

James Pustejovsky

Brandeis University
Department of Computer Science

CS 135: Computational Semantics

September 1, 2026

Where We Were

Last session. Meanings are functions; composition is β-reduction. We derived

$$[\![\text{Alice likes Bob}]\!] = \text{like}(\text{a}, \text{b})$$

in two steps, and the word-order problem from the opening slide dissolved.

But nothing stopped us writing nonsense. $[\![\text{Alice}]\!]([\![\text{Bob}]\!])$ is a perfectly well-formed untyped term. The calculus has no opinion about it.

Today's move

Assign every expression a **type**. Composition is licensed only when types fit. Ill-formed combination stops being *wrong* and becomes *impossible*.

Today

1. The type system: two basic types, one constructor
2. Types for lexical categories – and what the assignment commits us to
3. Composition: function application, and why types determine direction
4. Worked derivations, with trees
5. When application is not enough: predicate modification
6. Adjectives, and how they stress the system
7. Type mismatch, and type shifting
8. Models: what makes a sentence true
9. Where types break: quantification

Throughout: watch for the two things a type records – *arity* and *sort*. Theme 5 recovers one of them geometrically and struggles with the other.

Two Basic Types

Every expression of the language gets a **semantic type**. Two are basic:

Type	Denotes	Domain
e	an individual (*entity*)	$\mathcal{D}_e$ – the domain of discourse
t	a truth value	$\mathcal{D}_t = \{0, 1\}$

Examples.

- *Alice*, *Bob*, *the dog* – type e
- *Alice sleeps*, *Bob likes Alice* – type t

Proper names denote individuals. Sentences denote truth values.

Everything else in the language is a *function* built out of these two. That is a strong claim, and the rest of this session is about how far it gets us.

The Type Constructor

The type inventory, defined recursively

1. e and t are types.
2. If σ and τ are types, then $\langle \sigma, \tau \rangle$ is a type: the type of *functions from $\mathcal{D}_\sigma$ to $\mathcal{D}_\tau$*.
3. Nothing else is a type.

One base case, one constructor, one closure clause – the same shape as the definition of lambda terms last session, and the same shape as most definitions you will meet this semester.

The domains follow the types:

$$\mathcal{D}_{\langle \sigma,\tau \rangle} \;=\; \mathcal{D}_\tau^{\mathcal{D}_\sigma} \;=\; \{\, f \mid f : \mathcal{D}_\sigma \to \mathcal{D}_\tau \,\}$$

Note how fast this grows. If $|\mathcal{D}_e| = 4$, then $|\mathcal{D}_{\langle e,t \rangle}| = 2^4 = 16$, and $|\mathcal{D}_{\langle \langle e,t \rangle,t \rangle}| = 2^{16} = 65{,}536$.

Reading Type Notation

Types are right-associative and the brackets are often dropped:

$$\langle e, \langle e, t \rangle \rangle \quad \text{is often written} \quad e \to (e \to t) \quad \text{or just} \quad e \to e \to t$$

Read it as a recipe. $e \to e \to t$ says: *give me an individual, then another individual, and I will give you a truth value.*

Montague notation	**Arrow notation**	**Haskell (PS 1)**
e	e	`Entity`
t	t	`Bool`
$\langle e, t \rangle$	$e \to t$	`Entity -> Bool`
$\langle e, \langle e, t \rangle \rangle$	$e \to e \to t$	`Entity -> Entity -> Bool`
$\langle \langle e, t \rangle, t \rangle$	$(e \to t) \to t$	`(Entity -> Bool) -> Bool`

The third column is not an analogy. In Problem Set 1 the Haskell type *is* the semantic type, and the compiler enforces it.

The Type Hierarchy

Type	Denotes	Linguistic example
e	individuals	*Alice*, *the dog*
t	truth values	*Alice sleeps*
$\langle e, t\rangle$	sets of individuals	*sleeps*, *dog*, *tall*
$\langle e, \langle e, t\rangle\rangle$	binary relations	*likes*, *read*
$\langle e, \langle e, \langle e, t\rangle\rangle\rangle$	ternary relations	*gave*
$\langle\langle e, t\rangle, t\rangle$	sets of sets	*every student*, *someone*
$\langle\langle e, t\rangle, \langle e, t\rangle\rangle$	predicate modifiers	*quickly*, *former*
$\langle\langle e, t\rangle, \langle\langle e, t\rangle, t\rangle\rangle$	determiners	*every*, *some*, *no*
$\langle t, t\rangle$	truth-value operators	*not*
$\langle t, \langle t, t\rangle\rangle$	connectives	*and*, *or*

The bottom half is where things get interesting – and where the naive picture ("meanings are things and predicates apply to them") breaks down.

Types for Lexical Categories

Category	Type	Denotation
Proper name	e	a
Common noun	$\langle e, t\rangle$	$\lambda x.\,\mathsf{student}(x)$
Intransitive verb	$\langle e, t\rangle$	$\lambda x.\,\mathsf{sleep}(x)$
Transitive verb	$\langle e, \langle e, t\rangle\rangle$	$\lambda x.\lambda y.\,\mathsf{like}(y, x)$
Ditransitive verb	$\langle e, \langle e, \langle e, t\rangle\rangle\rangle$	$\lambda z.\lambda x.\lambda y.\,\mathsf{give}(y, x, z)$
Adjective (intersective)	$\langle e, t\rangle$	$\lambda x.\,\mathsf{tall}(x)$
Sentence	t	$\mathsf{like}(\mathsf{a}, \mathsf{b})$

Two things to notice.

- Common nouns and intransitive verbs **share a type**. Semantically, *dog* and *sleeps* are the same kind of object: a one-place predicate.
- The transitive verb takes its **object first**. This is a choice, and the next slides show why it is the right one.

Aside: Semantic Type is Not Syntactic Category

dog is a noun; *sleeps* is a verb. Syntactically they behave completely differently.

Semantically, both are functions from individuals to truth values.

	Syntactic category	**Semantic type**
dog	N	$\langle e, t \rangle$
sleeps	V	$\langle e, t \rangle$
tall	A	$\langle e, t \rangle$
asleep	A	$\langle e, t \rangle$

The map from categories to types is many-to-one. Semantics is coarser than syntax – it cares only about **what an expression combines with**, not how it is spelled or where it may occur.

This is why a semantic theory needs a syntax alongside it, related by a homomorphism.

Compositionality, Stated Precisely

Last session's informal version: *the meaning of the whole is determined by the meanings of the parts and how they are combined*.

The precise version:

Compositionality as homomorphism

There is a map $[\![\cdot]\!]$ from syntactic structures to meanings such that for every syntactic rule combining α and β into γ, there is a semantic operation F with

$$[\![\gamma]\!] = F([\![\alpha]\!], [\![\beta]\!])$$

Three consequences.

- Meaning attaches to *structures*, not strings. Ambiguous strings get several structures.
- F depends only on the *rule*, not on the particular words.
- List the rules and the lexicon, and you have specified infinitely many sentences.

The Composition Rule

Function Application (FA)

If α has type $\langle \sigma, \tau \rangle$ and β has type σ, then $\alpha(\beta)$ has type τ, and

$$[\![\alpha\,\beta]\!] \;=\; [\![\alpha]\!]([\![\beta]\!])$$

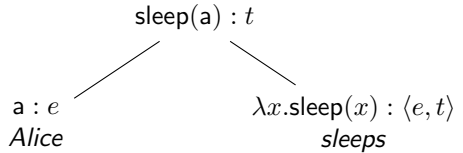
That is the *whole* composition rule for now. One rule.

And here is the key property. Given two expressions with known types, there is generally **only one way to combine them**. The types determine which is the function and which is the argument.

$\langle e, t \rangle$ applied to e ✓ $\qquad$ e applied to $\langle e, t \rangle$ ×

We never have to *say* that the verb applies to the subject. The type assignment already said it.

Derivation Tree: *Alice sleeps*



Reading the tree bottom-up:

1. The lexicon supplies a of type e and λx.sleep(x) of type $\langle e, t \rangle$.
2. Only one application typechecks: $\langle e, t \rangle$ takes e.
3. $(\lambda x.\text{sleep}(x))(\text{a}) \rightsquigarrow \text{sleep}(\text{a})$, of type t. ✓

We draw the subject on the left because that is the syntax. The semantics does not care – it reads the types.

Currying, and Why the Object Comes First

Recall from last session: the calculus has only one-argument functions, so a two-place relation is a function returning a function.

$$[\![\text{likes}]\!] = \lambda x.\lambda y.\ \text{like}(y, x)\ :\ \langle e, \langle e, t\rangle\rangle$$

The first argument x is the object; the second y is the subject. Why that order?

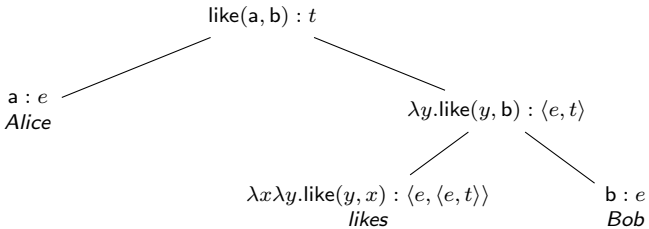
Because English builds the verb phrase first:

$$[_S\ \text{Alice}\ [_{VP}\ \text{likes Bob}\ ]\ ]$$

Applying to the object first gives us a constituent of type $\langle e, t\rangle$ – **exactly the type of an intransitive verb**. The VP *is* a one-place predicate, and the rest of the derivation cannot tell whether it came from *sleeps* or from *likes Bob*.

Partial application is phrase structure. This is not a coincidence – it is why the typed lambda calculus fits natural language as well as it does.

Derivation Tree: *Alice likes Bob*



Step 1. $(\lambda x.\lambda y.\,\text{like}(y, x))(\text{b}) \rightsquigarrow \lambda y.\,\text{like}(y, \text{b})$ – type $\langle e, t\rangle$

Step 2. $(\lambda y.\,\text{like}(y, \text{b}))(\text{a}) \rightsquigarrow \text{like}(\text{a}, \text{b})$ – type t ✓

And *Bob likes Alice* gives like(b, a). The two argument positions are never confused, at any depth of embedding.

Structure Matters: Bracketing

Compositionality assigns meaning to **structures**. Different bracketings give different meanings.

Alice gave Bob a book.

$$[\![\text{gave}]\!] = \lambda z.\lambda x.\lambda y.\ \text{give}(y, x, z) \ : \ \langle e, \langle e, \langle e, t\rangle\rangle\rangle$$

Three applications, in a fixed order:

$$\begin{aligned} [\![\text{gave a book}]\!] &= \lambda x.\lambda y.\ \text{give}(y, x, \text{bk}) && : \langle e, \langle e, t\rangle\rangle \\ [\![\text{gave Bob a book}]\!] &= \lambda y.\ \text{give}(y, \text{b}, \text{bk}) && : \langle e, t\rangle \\ [\![\text{Alice gave Bob a book}]\!] &= \text{give}(\text{a}, \text{b}, \text{bk}) && : t \end{aligned}$$

Each step strips one λ and lowers the type by one arrow.

Keep this shape in mind. In Session 18 the same derivation runs with grades 4, 3, 2, 1 in place of these types, and contraction in place of application.

When Application Is Not Enough

Consider *tall student*.

$$[\![\text{tall}]\!] : \langle e, t \rangle \qquad\qquad [\![\text{student}]\!] : \langle e, t \rangle$$

Neither can apply to the other. $\langle e, t \rangle$ applied to $\langle e, t \rangle$ does not typecheck – the function wants an e and is being handed a function.

Two possible responses:

1. Change the type of *tall* to $\langle \langle e, t \rangle, \langle e, t \rangle \rangle$, so it takes the noun as argument.
2. Keep both at $\langle e, t \rangle$ and add a second composition rule.

Both are used in the literature. We take (2) for now, and return to (1) when we look at adjectives more carefully.

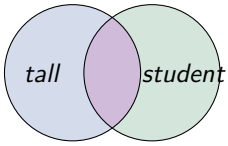
Predicate Modification

Predicate Modification (PM)

If α and β are both of type $\langle e, t\rangle$, then

$$[\![\alpha\, \beta]\!] \;=\; \lambda x.\, [\![\alpha]\!](x) \wedge [\![\beta]\!](x) \qquad : \langle e, t\rangle$$

Set-theoretically this is **intersection**: *tall student* denotes the tall things intersected with the students.



We now have two rules: FA and PM. You implement both in Problem Set 1. Keep them general – a composition rule that mentions a particular word is not a composition rule.

Adjectives Are Not All Alike

Predicate modification works for *tall student*. It fails badly elsewhere.

Class	**Example**	**Entailment pattern**
Intersective	*Norwegian student*	is Norwegian ✓ and is a student ✓
Subsective	*skillful surgeon*	is skillful ? and is a surgeon ✓
Non-subsective	*former senator*	is former × and is a senator ×
Privative	*fake diamond*	is a diamond ×

- *Norwegian student* = Norwegians ∩ students. PM is exactly right.
- *Skillful surgeon* is not skillful-things ∩ surgeons – skill is *relative to the activity*. A skillful surgeon may be a terrible violinist.
- *Former senator* is not in the set of senators at all. Intersection gives the wrong answer.

For the last three, the adjective must take the noun as its *argument*: type $\langle\langle e,t\rangle,\langle e,t\rangle\rangle$.

What the Adjective Data Tells Us

Class	Type	Rule
Intersective	$\langle e, t\rangle$	PM
Subsective	$\langle\langle e, t\rangle, \langle e, t\rangle\rangle$	FA
Non-subsective	$\langle\langle e, t\rangle, \langle e, t\rangle\rangle$	FA

An uncomfortable consequence. The same syntactic position – prenominal adjective – now corresponds to two different semantic types, and which applies depends on the *word*.

This is a genuine tension in the framework, not a detail. Either we complicate the type assignment, or we complicate the composition rules, or we let the lexicon carry the distinction. All three moves appear in the literature.

Generative Lexicon theory – which we return to in Session 10 – is one response: enrich what a lexical entry contains, so that the composition can be predicted rather than stipulated.

Type Mismatch Is Compositional Failure

What happens if we try to combine things that do not fit?

$$[\![\text{Alice}]\!]([\![\text{Bob}]\!]) \qquad — \qquad \text{type } e \text{ applied to type } e$$

There is no rule that licenses this. The derivation does not produce a wrong answer; **it produces no answer at all**.

The point

In a typed system, ill-formed combinations are not *false* – they are *undefined*. The type discipline is a theory of **what can compose with what**, and it is prior to any question of truth.

When you write the fragment in Problem Set 1, the Haskell type checker enforces exactly this. A type error is not an obstacle to the exercise. It is the exercise telling you that you have combined two meanings that cannot combine.

Sometimes Mismatch Should Be Repaired

Not every mismatch is an error. Consider:

Alice and every student left.

Alice is type e; *every student* is type $\langle\langle e,t\rangle,t\rangle$. Conjunction demands two things of the *same* type.

Partee's response (1987): allow expressions to **shift** type by general, meaning-preserving operations.

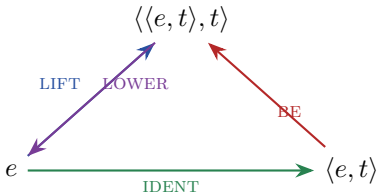
LIFT

$$\textsc{lift} \;:\; e \longrightarrow \langle\langle e,t\rangle,t\rangle \qquad \textsc{lift}(x) \;=\; \lambda P.\, P(x)$$

Alice becomes *the set of properties Alice has*. Now both conjuncts are $\langle\langle e,t\rangle,t\rangle$ and conjunction proceeds.

Note that $\textsc{lift}(x)$ applied to P β-reduces straight back to $P(x)$. Nothing is lost – the shift is a repackaging.

The Type-Shifting Landscape



Partee's triangle: three types that noun phrases can occupy, with systematic maps between them.

- LIFT: $x \mapsto \lambda P.P(x)$ – *Alice* as a generalized quantifier
- IDENT: $x \mapsto \lambda y.(y = x)$ – *Alice* as a predicate, for *That is Alice*
- LOWER: the partial inverse of LIFT

Hold onto this slide. These shifts are *discrete, stipulated* operations – a finite list of repairs. In Session 21 we ask whether a **rotor** could do this work continuously instead, with θ interpolating between the source and target readings.

What a Type Records

Step back. The type $\langle \sigma, \tau \rangle$ encodes **two independent things**:

	What it records	Example
Arity	how many arguments, in what order	*likes* takes 2
Sort	what *kind* each must be	*and* takes truth values

These come apart. $\langle e, t \rangle$ and $\langle t, t \rangle$ have the **same arity** and **different sorts** – and the type system keeps them apart.

A pointer to the end of the course

Theme 5 proposes that *grade* in a geometric algebra plays the role of type. But grade is a *number*: it records arity and nothing else. Whether sortal distinctions can be recovered geometrically – from basis structure, subspaces, or signature – is an open question, and a candidate final project.

From Formulas to Truth

We can now derive like(a, b) from *Alice likes Bob*. But is it *true*?

That question has no answer until we say what the symbols refer to.

A model $\mathcal{M} = \langle \mathcal{D}, I \rangle$

- $\mathcal{D}$ – a non-empty set, the **domain** of individuals
- I – an **interpretation function** assigning, to each constant, an object of the right type:
  - constants of type $e \mapsto$ members of $\mathcal{D}$
  - constants of type $\langle e, t \rangle \mapsto$ functions $\mathcal{D} \to \{0, 1\}$
  - and so on, by type

Denotation is always *relative to a model*: we write $[\![\alpha]\!]^{\mathcal{M}}$.

Meaning, in this tradition, is a systematic relation between expressions and a structure. The lambda calculus builds the expressions; the model supplies the world.

A Small Model

$$\mathcal{D} = \{\text{Alice}, \text{Bob}, \text{Carol}, \text{Fido}\}$$

Constant	**Interpretation**
a, b	Alice, Bob
student	{Alice, Bob, Carol}
dog	{Fido}
sleep	{Carol, Fido}
like	{(Alice, Bob), (Alice, Fido), (Bob, Fido)}

Now we can evaluate:

- $[\![\text{Alice likes Bob}]\!]^{\mathcal{M}} = \text{like}(\text{a}, \text{b}) = 1$ true
- $[\![\text{Alice sleeps}]\!]^{\mathcal{M}} = \text{sleep}(\text{a}) = 0$ false

This is essentially the model you will work with in Problem Set 1, slightly extended.

Predicates: Sets or Functions?

I have written sleep two ways – as a **set** $\{\text{Carol}, \text{Fido}\}$ and as a **function** of type $\langle e, t\rangle$. These are the same thing.

Characteristic function

A subset $S \subseteq \mathcal{D}$ corresponds to $\chi_S : \mathcal{D} \to \{0, 1\}$ with $\chi_S(x) = 1$ iff $x \in S$.

Set talk	**Function talk**	**Type**
$S \subseteq \mathcal{D}$	$\chi_S : \mathcal{D} \to \{0, 1\}$	$\langle e, t\rangle$
$R \subseteq \mathcal{D} \times \mathcal{D}$	curried $\mathcal{D} \to (\mathcal{D} \to \{0, 1\})$	$\langle e, \langle e, t\rangle\rangle$
$S \cap S'$	$\lambda x.\, \chi_S(x) \wedge \chi_{S'}(x)$	$\langle e, t\rangle$
$S \subseteq S'$	$\forall x.\, \chi_S(x) \to \chi_{S'}(x)$	t

Sets are easier to write down; functions compose. In Problem Set 1 the model is given as sets and the denotations as functions, and a helper converts between them.

Variables and Assignments

One loose end. What is the denotation of an expression containing a **free variable**?

$$[\![\text{sleep}(x)]\!]^{\mathcal{M}} = ?$$

There is no answer – x does not refer to anything until we say what it refers to.

Assignment function

An assignment g maps variables to objects of the appropriate type. Denotation is relative to a model *and* an assignment: $[\![\alpha]\!]^{\mathcal{M},g}$.

$$[\![x]\!]^{\mathcal{M},g} = g(x) \qquad [\![\lambda x.\alpha]\!]^{\mathcal{M},g} = \text{the function } d \mapsto [\![\alpha]\!]^{\mathcal{M},g[x \mapsto d]}$$

Here $g[x \mapsto d]$ is g modified to send x to d.

For closed terms – which is what our lexical entries are – the assignment makes no difference, and we suppress it. It becomes essential in Session 3, when quantifiers bind variables.

Practice

Using the model from two slides back, compute:

1. $[\![\text{Bob likes Fido}]\!]^{\mathcal{M}}$
2. $[\![\text{Carol sleeps}]\!]^{\mathcal{M}}$
3. The denotation of *sleeping student* (as a set)
4. What type does *likes Alice* have, and what is its denotation as a set?
5. Is *Alice likes Alice* true?

Practice

Using the model from two slides back, compute:

1. $[\![\text{Bob likes Fido}]\!]^{\mathcal{M}}$
2. $[\![\text{Carol sleeps}]\!]^{\mathcal{M}}$
3. The denotation of *sleeping student* (as a set)
4. What type does *likes Alice* have, and what is its denotation as a set?
5. Is *Alice likes Alice* true?

Answers.

1. $\text{like}(\text{b}, \text{Fido}) = 1$ true
2. $\text{sleep}(\text{Carol}) = 1$ true
3. $\{\text{Alice}, \text{Bob}, \text{Carol}\} \cap \{\text{Carol}, \text{Fido}\} = \{\text{Carol}\}$
4. Type $\langle e, t \rangle$; denotation $\{x : (x, \text{Alice}) \in \text{like}\} = \varnothing$ – nobody likes Alice here.
5. $(\text{Alice}, \text{Alice}) \notin \text{like}$, so false.

The Problem: Quantified Noun Phrases

Everything so far has been smooth. Here is where it stops.

Every student sleeps.

Alice sleeps worked because *Alice* is type e. So let us try type e for *every student*.

It fails, for at least three reasons:

- There is no *single individual* that *every student* refers to.
- *Every student sleeps* does not mean sleep(some particular person).
- *No student sleeps* would need to refer to a non-existent individual.

The Problem: Quantified Noun Phrases

Everything so far has been smooth. Here is where it stops.

Every student sleeps.

Alice sleeps worked because *Alice* is type e. So let us try type e for *every student*.

It fails, for at least three reasons:

- There is no *single individual* that *every student* refers to.
- *Every student sleeps* does not mean sleep(some particular person).
- *No student sleeps* would need to refer to a non-existent individual.

And a fourth, decisive one: *Every student sleeps* and *Alice sleeps* have the same syntax. If the semantics is compositional, they must combine by the same rule.

Montague's Resolution

If *every student* cannot be the argument, make it the **function**.

$$[\![\text{every student}]\!] \;=\; \lambda P.\, \forall x\, [\, \text{student}(x) \to P(x)\,] \qquad : \langle\langle e, t\rangle, t\rangle$$

A **generalized quantifier**: a function from predicates to truth values. Equivalently, a *set of sets* – the sets that contain all the students.

Now composition runs the other way:

$$[\![\text{Every student sleeps}]\!] = [\![\text{every student}]\!]\big([\![\text{sleeps}]\!]\big) \rightsquigarrow \forall x\, [\, \text{student}(x) \to \text{sleep}(x)\,]$$

Type t. ✓ And the composition rule is still just FA.

The type determined which expression is the function. That is the deepest lesson of this session: we do not stipulate the direction of application; the type assignment does it for us.

And the Determiner Itself

If *every student* is $\langle\langle e,t\rangle,t\rangle$ and *student* is $\langle e,t\rangle$, then *every* must be

$$[\![\text{every}]\!] = \lambda Q.\lambda P.\ \forall x\,[\,Q(x) \rightarrow P(x)\,] \qquad : \langle\langle e,t\rangle,\langle\langle e,t\rangle,t\rangle\rangle$$

A function from a predicate (the **restrictor**) to a generalized quantifier.

Determiner	**Denotation**
every	$\lambda Q.\lambda P.\ \forall x\,[\,Q(x) \rightarrow P(x)\,]$
some, a	$\lambda Q.\lambda P.\ \exists x\,[\,Q(x) \wedge P(x)\,]$
no	$\lambda Q.\lambda P.\ \neg\exists x\,[\,Q(x) \wedge P(x)\,]$

You will implement all three in Problem Set 1. Note that *no* can be defined in terms of *some* – and that the pattern $\lambda Q.\lambda P$ is the same in all three; only the logical skeleton changes.

What This Sets Up

Generalized quantifiers are the price of compositionality – and they buy something expensive.

Every student read a book.

Two quantifiers in one sentence. Which takes scope over which?

- **Surface** ($\forall > \exists$): each student read some book, possibly different ones.
- **Inverse** ($\exists > \forall$): one particular book was read by every student.

Two readings, same words, same syntax. The types alone do not tell us which derivation to run.

Session 5 is about this – and it is the capstone of Theme 1, because scope is where composition stops being mechanical and starts requiring a theory.

Summary

Concept	**Key point**
Basic types	e (individuals), t (truth values)
Type constructor	$\langle \sigma, \tau \rangle$, applied recursively; one rule builds all types
What a type records	*arity* and *sort* – and these come apart
Function application	the composition rule; types determine its direction
Predicate modification	for two $\langle e, t \rangle$ meanings; set intersection
Adjective classes	intersective / subsective / non-subsective need different types
Type mismatch	not falsehood – *undefinedness*. Composition simply fails
Type shifting	LIFT, IDENT, LOWER: discrete, stipulated repairs
Model $\langle \mathcal{D}, I \rangle$	supplies the world; denotation is relative to it
Assignment g	needed once variables are free; essential from Session 3
Generalized quantifiers	NPs of type $\langle \langle e, t \rangle, t \rangle$; the NP becomes the function

Three Threads to Carry Forward

1. **Types determine direction.** We never stipulate which expression applies to which. This is the property Theme 5 must reproduce if grade is to do the work of type.
2. **Arity and sort are different.** Grade is a number and records only arity. Whether sortal distinctions survive the move to geometry is open.
3. **Coercion is currently a stipulation.** Partee's shifts are a finite list of repairs applied when composition fails. Session 21 asks whether a rotor could do this continuously instead.

Nearly everything in today's session has a counterpart in Sessions 17–21. That is by design – you cannot evaluate the geometric proposal without owning the logical original.

Coming Up

Session 3 (Friday): Predicate logic and models

- Quantifiers $\forall$ and $\exists$ in the object language
- Satisfaction, assignment functions, validity
- Evaluating formulas against a model, systematically
- **Problem Set 1 goes out**

Reading: EU 3.6–3.9.

Reminders

- Linear algebra self-assessment: report your six section scores via the course-site form by **tonight**.
- The answer key and review guide are posted. If a section came out weak, the guide says exactly what to review – and Weeks 2–5 are when there is room to do it.

Questions?

Next: Session 3 — Predicate Logic and Models